Failure (a bad split, and a k comparison) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Two functions carry this exercise: a chunker, and a retriever.

`chunk_by_chars(text: str, chunk_size: int) -> list[str]` is a deliberately "dumb" splitter — it cuts the string every `chunk_size` characters, no matter what's there.

`retrieve(question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int) -> list[tuple[float, str]]` embeds the question once, scores every stored `(chunk_text, embedding)` pair against it with cosine similarity, sorts by score descending, and returns the top `k`.

The exact pieces:

- A `while start < len(text):` loop slicing `text[start:start+chunk_size]`, moving `start` forward by `chunk_size` each time.
- `time.perf_counter()` — a monotonic clock, better than `time.time()` for measuring short durations.
- `list[tuple[str, list[float]]]` for storage — a plain Python list of `(chunk_text, embedding)` pairs is enough at this scale; no real vector database needed.

The timing you did above barely moves between `k=1`, `k=3`, and `k=10` — and that result is easy to over-generalize from. It only barely moves because `retrieve()` here compares your question against a *handful* of chunks, one by one, in a plain Python loop. That's brute-force search: cost grows in direct proportion to how many chunks exist, regardless of `k`. At a few dozen chunks, brute-force is effectively free. At a few hundred thousand — a real company's documentation, ticket history, or knowledge base — comparing against every single one for every single question stops being free, and a real vector database (Chroma, FAISS, a hosted vector store) uses an *approximate* nearest-neighbor index instead: a data structure built in advance that finds very-likely-closest chunks in roughly constant time, without touching every stored vector for every query. This document's Core

Concepts describes this as the actual job of a "vector store" — this exercise's brute-force loop is a stand-in for it, not the real thing.

The second thing worth separating out: fixing the "half the fact went missing" bug you caused on purpose here is not the same problem as picking a good `k`. `k` decides how many results to return; where the boundary falls is decided by the chunker. Raising `k` from 1 to 10 hides the symptom for this one tiny document (where there's nowhere else for the missing half to hide) — it does not fix the cause, and won't reliably help at all once the corpus is bigger (see `chunk_boundary_split`'s solution for why). The actual fix is chunking with overlap, applied *before* any of this timing comparison happens.

The extra pieces:

- Measure `retrieve()`'s timing again, but against a much larger synthetic chunk list (repeat/pad your test document hundreds of times) — watch the per-query time actually start to grow with corpus size, unlike the near-flat numbers from a few dozen chunks. Embed those hundreds of chunks in **batches** (`input=` a list of up to 2,048 texts per call), not one call per chunk — the `get_embeddings_batch` idea from the Basic exercise.
- Re-run the whole exercise using `chunk_by_chars_with_overlap` (from `chunking_methods`, Intermediate section) instead of the plain `chunk_by_chars`, and confirm `k=1` alone now returns the complete fact. The rule: any text up to `overlap` characters long always fits whole inside some chunk — the fact sentence is 71 characters, so use `chunk_size=120, overlap=80`.

Hint 2 — The plan, and almost the whole thing

```
# chunking_practice.py — Failure section
import math
import time
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text: str, chunk_size: int) -> list[str]:
    chunks: list[str] = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size
    return chunks

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
```

```
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def retrieve(
    question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int
) -> list[tuple[float, str]]:
    question_embedding = get_embedding(question)
    scored: list[tuple[float, str]] = []
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]
```

What's missing: `TEST_DOCUMENT`, a `main()` that builds `chunks_with_embeddings`, prints the chunks, runs `retrieve()` at `k=1` alone, then loops over `k` in `(1, 3, 10)` timing each call with `time.perf_counter()`. Write that yourself, then compare against the Solution.

Once that's working, add the missing variable — corpus size — and the real chunking fix:

```
build a much bigger chunk list: your TEST_DOCUMENT repeated/padded until
    there are hundreds (or low thousands) of chunks
embed them in slices of 1,000 per embeddings call, not one call each

time retrieve() against this bigger list at the same k values (1, 3, 10)
    -> compare against the timings from the small chunk list

separately: build chunks_with_overlap using chunk_by_chars_with_overlap
    instead of chunk_by_chars, on the ORIGINAL small TEST_DOCUMENT,
    with chunk_size=120, overlap=80 (overlap >= the 71-char fact)
```

```
run retrieve() at k=1 against the overlapping chunks
    -> does the single top chunk now contain the whole fact?
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py — Failure section
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int, overlap: int,
) -> list[str]:
    chunks: list[str] = []
    step = chunk_size - overlap
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        # your turn: advance start by step, not chunk_size
        ...
    return chunks

def build_large_corpus(document: str, repeats: int) -> str:
    # your turn: return `document` repeated `repeats` times, so you have a
    # much bigger (but still synthetic) document to chunk and time against
    ...
```

Fill in both functions, run the timing comparison at small vs. large corpus size, and run the overlap comparison at `k=1`, then compare all of your finished versions against the Solution.

Full solution: Show me the solution